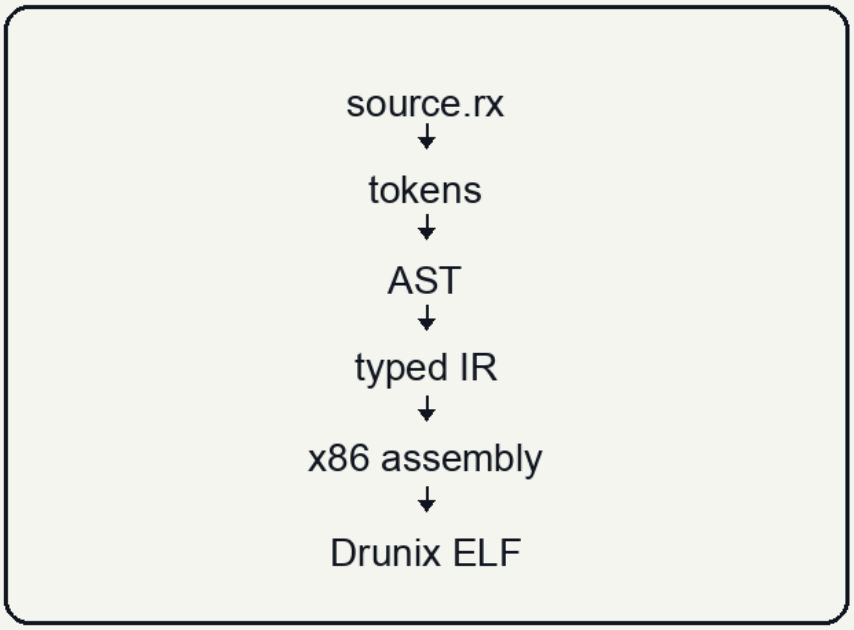
Rexy

Building a Systems Language Compiler for Drunix



My journey in writing a compiler for my OS

A companion to the Drunix book

Drew Tennenbaum

2026

Contents

Part I - Getting Started With Rexy 1
- Chapter 1 - The First Rexy Program 2
- Chapter 2 - Values, Types, and Casts 4
- Chapter 3 - Bindings and Functions 6

Part II - Control Flow 7
- Chapter 4 - Conditions and Loops 8
- Chapter 5 - Pattern Matching with `match` 10

Part III - Building Data 11
- Chapter 6 - Structs 12
- Chapter 7 - Enums and Variants 14
- Chapter 8 - Tuples and Slices 16

Part IV - Memory and the `unsafe` Boundary 17
- Chapter 9 - Pointers and Statics 18
- Chapter 10 - The `unsafe` Boundary 20

Part V - Organising Programs 21
- Chapter 11 - Modules and Visibility 22
- Chapter 12 - Generics 24

Part VI - The Standard Library and Drunix 26
- Chapter 13 - A Tour of the Standard Library 27
- Chapter 14 - Building Programs for Drunix 29

Front Matter

Part I - Getting Started With Rexy

Before we can talk about what makes Rexy interesting, we need a program that runs. Part I gets the reader from "I have never written a Rexy file" to "I can write small Rexy programs that compute and return a value".

Chapter 1 walks through the smallest possible Rexy program, builds it, and explains what each piece is doing. Chapter 2 introduces Rexy's primitive values: integers, booleans, characters, strings, and how to convert between them. Chapter 3 covers bindings, functions, and the rules Rexy enforces about mutation and types.

By the end of Part I, the reader can write a Rexy file with multiple functions, declare local variables, mutate them where needed, and return a result. That is the floor we will build everything else on.

Chapter 1 - The First Rexy Program

The Smallest Program That Runs

Every language has a smallest program that does something observable. In Rexy, that program is a single function:

```
fn main() -> i32 {
    return 0;
}
```

Save it as `hello.rx`. The `.rx` extension is what the compiler looks for. The file holds one **item**, which is Rexy's word for the things that can sit at the top level of a source file: functions, structs, enums, statics, modules, and a few others. We will meet each of those in time. For now we only need a function.

`fn` introduces a function definition. The name `main` is special: it is the entry point Rexy expects every executable program to provide. The `-> i32` says that this function returns a 32-bit signed integer. The body is delimited by braces, and inside the braces, `return 0;` hands a value back to whoever called the function.

When this program runs on Drunix, the operating system reads that integer as the program's exit status. Returning `0` is the conventional way to say "the program finished without an error". Any other value signals a problem the caller might want to know about.

Building It

Once `hello.rx` exists, the compiler turns it into something Drunix can run. The exact commands belong in the final chapter, where we cover the full path from source to executable. For now, it is enough to know that `rexc` is the compiler and that it consumes `.rx` files.

We will run programs throughout the book. When we do, the question we will ask is not "what did the compiler do?" but "what did the program compute?". Rexy is small enough that we can usually answer that by reading the source.

Statements End in Semicolons

Notice the semicolon at the end of `return 0;`. Rexy is a statement-oriented language: the body of a function is a sequence of statements, each terminated by a semicolon. A statement is an instruction the program executes in order. We will meet the full statement list as we go: `let`, `return`, assignment, calls, `if`, `match`, `while`, `for`, `break`, `continue`, and a few others.

Braces group statements into a **block**. A function body is one block. So are the bodies of `if`, `while`, `for`, and `match` arms. Blocks introduce their own scope, which we will care about as soon as we have local variables to talk about.

A Slightly Bigger First Program

Returning a constant is not very interesting. We can do a little more before we even leave Chapter 1:

```
fn main() -> i32 {
    return 1 + 2;
}
```

The expression `1 + 2` is evaluated, the result is `3`, and `3` is what the program returns. Expressions are the right-hand side of statements. They produce values. Rexy's expression grammar covers the things any C-family programmer expects: integer literals, arithmetic, comparison, boolean operators, parentheses, function calls, and a handful of other forms we will introduce as we need them.

Where We Are by the End of Chapter 1

We have written a Rexy file. We know that every program needs a `main` function returning `i32`. We know that statements end in semicolons, that blocks are delimited by braces, and that the value `main` returns becomes the program's exit status on Drunix.

That is enough vocabulary to start asking what kinds of values Rexy can hold and how the language describes them. Chapter 2 takes that question on directly.

Chapter 2 - Values, Types, and Casts

Why Types Are Written Down

Rexy is a typed language with no inference at the binding level. Every local variable, every function parameter, every return type, and every static is written down in the source. That is a deliberate cost. It buys two things that matter for a systems language: the reader of a program never has to guess what a name holds, and the compiler never has to guess either.

Types in Rexy describe what a value *is*, including how many bits it occupies, whether it is signed, and whether the language treats it as a number, a flag, a code point, or a string.

The Primitive Types

Rexy ships with a small, regular primitive set:

Family	Types	Notes
Signed integers	`i8`, `i16`, `i32`, `i64`	Two's-complement, by width in bits.
Unsigned integers	`u8`, `u16`, `u32`, `u64`	Same widths, no sign bit.
Boolean	`bool`	Holds `true` or `false`.
Character	`char`	A single ASCII code point.
String	`str`	An immutable string slice.

Signed and unsigned integers are different types. `i32` and `u32` both occupy 32 bits, but the language refuses to mix them in arithmetic without an explicit conversion. Comparisons and division behave differently for signed and unsigned values, and Rexy does not let those differences hide.

Integer Literals

Integer literals are written in decimal and have no implicit type until they are placed into a context that supplies one:

```
let small: i32 = 42;
let big: u64 = 18446744073709551615;
let negative: i64 = -9223372036854775808;
```

Each literal is checked against the type it lands in. The compiler keeps the original decimal text of every literal so it can validate values that are too large to fit in an intermediate signed integer during parsing. A literal that will not fit in its declared type is rejected with a diagnostic at the literal's source location.

Booleans

`bool` has exactly two values, written `true` and `false`. Boolean values are not integers, and the language will not let us treat them as numbers. The condition of an `if`, a `while`, or a `for` must be a `bool`, not "anything that is not zero". That rule is the same idea as written-down types: the language refuses to guess what we meant.

Characters and Strings

`char` is a single character literal, written between single quotes:

```
let letter: char = 'A';
let newline: char = '\n';
```

Character literals support a small set of escapes: `\n`, `\r`, `\t`, `\'`, `\"`, and `\\`. Each escape produces one character.

`str` is an immutable string, written between double quotes:

```
let greeting: str = "hello";
```

Strings carry the same escapes as characters. We will say more about how strings are represented when we reach pointers and slices, but for everyday use a `str` is just a value we can pass around and hand to print functions.

Casting With `as`

Rexy converts between primitive types only when we ask it to. The conversion operator is `as`:

```
let code: u32 = 'A' as u32;
let truncated: u8 = 300 as u8;
let widened: i32 = (truncated as i32) + 1;
let flag: i32 = true as i32;
```

`as` performs the obvious primitive conversion: integers widen or narrow by copying or truncating bits, signed integers extend their sign on widening, unsigned integers fill with zeros, `char` values convert to the integer of their code point, and `bool` converts to `0` or `1`. The rule is that every conversion is local and visible: a reader can see exactly where a value changed type.

Where We Are by the End of Chapter 2

We have a vocabulary of values. We can write integer, boolean, character, and string literals. We can declare a local with a type annotation, even if we have not yet pinned down all the rules around what a `let` binding really means. We can convert between primitive types with `as` whenever the program needs to.

What we cannot do yet is compose those values into something a program can *do*. Chapter 3 introduces bindings and functions, the two constructs that turn isolated values into a working program.

Chapter 3 - Bindings and Functions

`let` Introduces a Local

A **binding** is a name attached to a value inside a function body. Rexy introduces bindings with `let`:

```
let count: i32 = 0;
```

Every part of that line is required. The keyword `let` says we are introducing a new local. The name `count` is the identifier we will use to refer to the value. The type annotation `: i32` tells the compiler what the binding holds. The right-hand side `= 0` is the initial value. The semicolon ends the statement.

Type annotations are not optional. Rexy does not attempt to infer the type of a binding from its initial expression. The trade-off is the one we discussed in Chapter 2: a small amount of typing now buys a program that any reader can follow without running the compiler in their head.

Mutation Is Opt-In

A plain `let` binding cannot be reassigned. The value it holds at the moment of declaration is the value it holds for the rest of the function. To create a binding that can be reassigned, we mark it `mut`:

```
let mut count: i32 = 0;
count = count + 1;
count = count + 1;
```

Without `mut`, the second and third lines would be rejected. That refusal is the language's quiet way of pointing out that mutation is a separate capability from existence. Most locals do not need it. The ones that do are labelled, so a reader scanning a function body can see at a glance which names change over the course of the function.

Reassignment uses ordinary syntax: `count = count + 1;`. The form is the same whether we are incrementing, recomputing, or replacing.

There are also two compact mutation forms inherited from the C family. `++` and `--` increment and decrement an integer binding by one, either as a prefix or a postfix:

```
let mut i: i32 = 0;
++i;
i++;
```

Both produce the same effect on `i`. We will use them inside loops where they are conventional and read smoothly.

Functions

We have already seen the shape of a function from Chapter 1. The general form is:

```
fn name(parameter_list) -> return_type {
    statements
}
```

A function takes a comma-separated list of parameters, each of which is a name and a type, and returns a value whose type is named after the `->`. The body is a block of statements.

```
fn add(a: i32, b: i32) -> i32 {
    return a + b;
}

fn double(value: i32) -> i32 {
    return add(value, value);
}
```

Calling a function uses ordinary parentheses, with positional arguments. The compiler checks that we passed the right number of arguments, that each one has the type the parameter declared, and that the call site uses the return value in a context that matches its type.

`return` Is a Statement

Every function in Rexy must end its execution with a `return` statement. The expression after `return` becomes the function's result. This is not the same as some languages where the last expression of a block is implicitly returned. Rexy is explicit: the only way a value leaves a function is through a `return`.

When a function has multiple paths, every path must end in a `return`:

```
fn sign(value: i32) -> i32 {
    if value > 0 {
        return 1;
    } else if value < 0 {
        return -1;
    } else {
        return 0;
    }
}
```

If a path could fall off the end of a function without returning, the compiler rejects the program. We never wonder what a Rexy function returned. If it returned at all, it returned because of a `return`.

Parameter and Return Types Cover the Whole Surface

A function's signature is its full external description. The type of every parameter is in the signature. The return type is in the signature. There are no implicit parameters, no hidden return paths, and no values that exist outside the type system. When we read a Rexy function header, we are reading the entirety of its contract with the rest of the program.

Where We Are by the End of Chapter 3

We can declare local variables with `let`, mark them `mut` when we need to reassign them, increment and decrement integers, define functions with typed parameters and a typed return, and call functions positionally. We have the ingredients for a useful straight-line program.

What we still lack is the ability to choose between paths and to repeat work. Part II covers both: Chapter 4 brings in conditions and loops, and Chapter 5 brings in `match`, which is how Rexy expresses choice over structured data.

Part II - Control Flow

A straight-line program runs the same statements in the same order every time. Real programs choose, repeat, and respond to data. Part II covers the two ways Rexy expresses that.

Chapter 4 introduces the C-family side of Rexy's control flow: `if` and `else` for choice, `while` and `for` for repetition, and `break` and `continue` for loop control. These are the constructs we reach for when the condition is a simple boolean test. Chapter 5 covers `match`, the structured form of choice. `match` becomes essential the moment we have enums and struct-shaped data, but it is a useful tool over plain integers and characters from the start.

By the end of Part II, the reader can write programs that branch on conditions, iterate over a counter, and dispatch on the value or shape of a piece of data. Together with bindings and functions, that is enough to express any computation we want to express, even before we have composite types.

Chapter 4 - Conditions and Loops

`if` and `else`

The simplest form of choice is `if`:

```
if value > 0 {
    return 1;
}
```

The condition between `if` and the opening brace must be a `bool`. Rexy will not silently treat an integer as a condition the way C would. If we want to test whether `value` is non-zero, we write `value != 0` and let the comparison produce a `bool`.

`if` accepts an optional `else`, and the `else` may be either another block or another `if`:

```
if value > 0 {
    return 1;
} else if value < 0 {
    return -1;
} else {
    return 0;
}
```

There is no separate ternary operator. The chain of `if/else if/else` covers every case.

Boolean Operators

The comparison and boolean operators are the ones a C-family programmer expects: `==`, `!=`, `<`, `<=`, `>`, `>=` for comparisons, and `&&`, `||`, `!` for logical combination. `&&` and `||` short-circuit: the right operand is only evaluated if the left operand did not already determine the result.

```
if !right && (left || right) {
    return 1;
}
```

The full precedence is the conventional one. Comparisons bind tighter than `&&`, which binds tighter than `||`. Parentheses always work and are encouraged whenever the precedence is not immediately obvious.

`while` Loops

`while` repeats a block as long as a `bool` condition holds:

```
let mut value: i32 = 0;
while value < 10 {
    value = value + 1;
}
```

The condition is checked before each iteration, including the first. If the condition is `false` to start with, the loop body never runs.

`for` Loops

Rexy's `for` loop is the C-style three-part form: an initializer, a condition, and an increment.

```
for let mut i: i32 = 0; i < 8; i = i + 1 {
    // body
}
```

The initializer can be a `let` statement or an assignment to an existing binding. The condition is a `bool` expression. The increment is an assignment, an indirect assignment through a pointer, or one of the `++` and `--` increment forms. The parts may also be wrapped in parentheses if that reads better:

```
for (let mut i: i32 = 0; i < 8; ++i) {
    // body
}
```

Use whichever form fits the surrounding code. They produce the same loop.

`break` and `continue`

Inside a `while` or `for`, `break` exits the loop immediately and `continue` skips the rest of the current iteration:

```
let mut value: i32 = 0;
while value < 10 {
    value = value + 1;
    if value == 3 {
        continue;
    }
    if value == 7 {
        break;
    }
}
```

When the loop above exits, `value` holds `7`. We saw `value` reach `3`, skipped the rest of that iteration, kept counting, and then `break` ended the loop the moment `value` hit `7`.

`break` and `continue` only make sense inside a loop. Using them outside one is a diagnostic, not a runtime error.

Conditions Stay Boolean

It is worth saying once more, because it is the most common surprise for a programmer arriving from C: every condition in Rexy must already be a `bool`. There is no implicit conversion from integer to boolean. If we want "non-zero", we write `!= 0`. If we want "non-empty string", we write a call to a string predicate. The language always asks: what does this condition *mean*?

Where We Are by the End of Chapter 4

We can branch on conditions, loop while a condition holds, count through a range with a `for`, and use `break` and `continue` to shape the way a loop ends or skips iterations. Combined with bindings and functions, that is enough machinery to write almost any algorithm we have in mind, as long as the data we are working over is simple values.

When the data has shape, plain `if` chains become awkward. Chapter 5 introduces `match`, the construct Rexy gives us for working with structured data once we have it.

Chapter 5 - Pattern Matching with `match`

When `if` Chains Stop Helping

Suppose a function takes an integer command code and dispatches on it:

```
fn handle(code: i32) -> i32 {
    if code == 0 {
        return 100;
    } else if code == 1 || code == 2 {
        return 101;
    } else {
        return -1;
    }
}
```

That works, but the structure is hidden in repetition. Each arm restates `code ==`. The fallback case sits at the end as an `else`. The relationship between the cases is not visible at a glance.

Rexy provides a structured form for this kind of dispatch: `match`.

The Shape of a `match`

A `match` chooses one block to execute based on the shape of a value:

```
match code {
    0 => { return 100; },
    1 | 2 => { return 101; },
    _ => { return -1; },
}
```

The expression after `match` is the value we are testing. Inside the braces is a list of **arms**. Each arm has the shape `pattern => block`. The first arm whose pattern matches the value is the arm that runs.

Arms are separated by commas, and a trailing comma after the last arm is allowed. Each arm's body is a block, even when that block is a single statement. The block contains the same kind of statements we use anywhere else.

Patterns

The simplest patterns are literals: integers, booleans, characters. They match a value if the value is equal to the literal:

```
match c {
    'a' | 'e' | 'i' | 'o' | 'u' => { return 1; },
    _ => { return 0; },
}
```

The `|` between two patterns is an alternation. An alternation arm matches if any of the listed patterns match. The `_` pattern is a wildcard. It matches any value and is how we express "everything else".

Negative integer literals work too, so we can match a signed integer directly:

```
match value {
    -1 => { return 0; },
    0 => { return 1; },
    1 => { return 2; },
    _ => { return 3; },
}
```

Once we introduce structs and enums in Part III, patterns will grow to include constructor patterns that destructure variants and struct patterns that bind fields by name. We will revisit `match` then with those patterns in hand. For now, literals and the wildcard cover everything we need.

`match` is a Statement

`match` is a statement, not an expression. It does not produce a value the way a `match` does in some other languages. To get a value out of a `match`, each arm `return`s, or each arm assigns to a `mut` binding declared before the `match`:

```
let mut label: i32 = 0;
match value {
    0 => { label = 100; },
    1 | 2 => { label = 101; },
    _ => { label = -1; },
}
```

Both styles are reasonable Rexy. The `return`-from-each-arm style suits a function whose only job is to dispatch. The "assign into a `mut` binding" style suits a function that needs the value for further work.

Why Coverage Matters

A `match` statement should account for every value the matched type can hold. For integers, the wildcard covers the cases we did not list. For booleans, the only legal values are `true` and `false`. Once we have enums, the compiler can tell us when we forgot a variant, and that will be one of the places `match` earns its keep most clearly.

Where We Are by the End of Chapter 5

We have two complementary tools for control flow. `if` chains handle conditions where the condition is a boolean computation. `match` handles choice over the value or shape of a data piece. Together they cover the cases the programs we will write actually need.

What we do not yet have is the data those programs will operate on. Part III introduces structs, enums, tuples, and slices. Once we have composite types, every construct from Part II becomes much more useful.

Part III - Building Data

The values we have seen so far are isolated: a number, a boolean, a character. Every interesting program eventually wants to put values together into something with shape. Part III covers the four ways Rexy lets us do that.

Chapter 6 introduces **structs**, named records of fields. Chapter 7 introduces **enums**, tagged unions whose variants can carry data. Chapter 8 covers two smaller composites: **tuples**, which group a fixed number of typed values without naming the fields, and **slices**, which give us a uniform way to refer to a contiguous run of values of the same type.

By the end of Part III, the reader can model the data their program is about. Functions can take structs and enums as arguments, return them as values, and `match` over their variants and fields. The language goes from "a pile of integers" to "a pile of named, structured values".

Chapter 6 - Structs

Naming a Group of Values

A **struct** is a named record of fields. Each field has its own name and its own type. Structs let us bundle related values into a single thing the program can pass around, store, and reason about as one unit.

We declare a struct at the top level of a file, alongside functions:

```
struct Point {
    x: i32,
    y: i32,
}
```

The declaration creates a new type called `Point`. From this point on, `Point` may appear anywhere a primitive type may appear: in a parameter list, as a return type, or as the type of a `let` binding.

Field declarations are separated by commas. A trailing comma after the last field is allowed. The compiler does not care about whether the fields are on their own lines, but the convention in this book is one field per line for anything beyond the smallest structs.

Building a Struct Value

To build a value of a struct type, we use a **struct literal**:

```
let origin: Point = Point { x: 0, y: 0 };
```

The literal names the struct, then provides each field by name. Every field the struct declares must appear, and a field name may not appear twice. The order does not have to match the declaration order, but readers will thank us if it usually does.

Struct values can be passed to functions like any other value:

```
fn shifted(p: Point, dx: i32) -> Point {
    return Point { x: p.x + dx, y: p.y };
}
```

The return type of `shifted` is `Point`. Inside the body, `p.x` and `p.y` read the fields of the parameter, and the return statement constructs a fresh `Point`.

Reading Fields

Field access uses a dot:

```
let p: Point = Point { x: 3, y: 4 };
let x: i32 = p.x;
let y: i32 = p.y;
```

A field expression has the type of the field. The same dot syntax works inside any larger expression. There is nothing magical about it: it is just the way to ask a struct value for one of its parts.

Mutating Fields Through a Pointer

Fields of a local struct can be re-bound by assigning a new struct value to the local. To mutate a field of a struct that is referenced through a pointer, Rexy provides a dedicated form. We will cover pointers in detail in Chapter 9, but the syntax is worth seeing now:

```
unsafe {
    (*p).x = 10;
}
```

That is the form Rexy uses to write through a pointer to a struct field. The parentheses and the leading `*` make the indirection explicit, which is the same explicitness theme we have followed since Chapter 2: when we change something, the code shows where and how.

Public Structs

A struct declared inside a module is private to that module by default. To let other modules construct it and read its fields, we mark it `pub`:

```
pub struct Point {
    x: i32,
    y: i32,
}
```

Visibility becomes interesting once we have more than one module. Chapter 11 covers modules and the visibility rules around them in full. For now it is enough to know that the keyword exists and that we will use it where needed.

Where We Are by the End of Chapter 6

We can declare a struct, build values of that struct, read its fields, and hand it to functions. We have a clean way to give names to grouped data.

What structs cannot do is express choice. A `Point` is always an `x` and a `y`. There is no way to say "this is *either* a point *or* a label". For that we need enums. Chapter 7 adds them, and once both are in our toolbox we can model the data of almost any program we will write.

Chapter 7 - Enums and Variants

Choice as a Type

A **struct** is a record of fields, all present at once. An **enum** is the opposite shape: exactly one of several alternatives. Enums let us write types like "either a success with an integer or an error with a code" and have the compiler enforce that we handle both cases.

The simplest enum is a list of named alternatives:

```
enum Direction {
    North,
    South,
    East,
    West,
}
```

Each name in the body is a **variant**. A value of type `Direction` is exactly one variant, never zero, never more than one. The compiler tracks which variant a value holds and uses that tracking to check `match` statements.

Variants With Payloads

Variants may carry data of their own. A variant followed by a parenthesized type list becomes a tagged record:

```
enum Outcome {
    Ok(i32),
    Err(i32),
}
```

A value of type `Outcome` is either `Ok(value)` for some `i32` value or `Err(code)` for some `i32` code. Different variants of the same enum can carry different types and different numbers of fields. Variants without a payload, like `North` above, are simply variants whose payload list is empty.

Building an Enum Value

To build a value of an enum type, we name the enum and the variant with `::` and supply the payload, if any:

```
let win: Outcome = Outcome::Ok(42);
let lose: Outcome = Outcome::Err(1);
let way: Direction = Direction::North;
```

The path `Outcome::Ok` is a qualified name. We will see `::` again in modules, where the same operator separates module names. For enum construction, the rule is simply: enum name, then `::`, then variant name.

Matching on Variants

Enums become useful the moment we `match` on them. The pattern syntax we saw in Chapter 5 grows two new forms here. A constructor pattern matches a specific variant and binds its payload:

```
match result {
    Outcome::Ok(value) => {
        return value;
    },
    Outcome::Err(_) => {
        return -1;
    },
}
```

In the first arm, `value` is a fresh local that holds the `i32` carried by the `Ok` variant. It is in scope for the body of that arm only. In the second arm, the wildcard _ says we do not need the payload, only the variant.

For variants whose payload looks like a struct, a struct-shaped binding list inside braces lets us name each field individually. Most of the enums we write will use the parenthesized payload form, but the brace form exists for the cases where naming each field reads better than positional binding.

Why `match` Earns Its Keep Here

When the value being matched is an enum, Rexy can tell us about variants we forgot. If `Outcome` grows a third variant later, every `match` over an `Outcome` that does not handle the new variant becomes a diagnostic at the point where the `match` was written. That is the property `if`/`else if` chains cannot give us: enums plus `match` make new alternatives a checked change, not a silent one.

Public Enums

Like structs, an enum declared in a module is private by default. To make the type and its variants visible to other modules, we mark the enum `pub`:

```
pub enum Outcome {
    Ok(i32),
    Err(i32),
}
```

The visibility applies to the enum and all of its variants together. There is no way to expose one variant without exposing the others, and that is deliberate: variants only make sense as a set.

Where We Are by the End of Chapter 7

We can model choice as a type. We can attach data to a variant and pull that data back out with a `match`. Combined with structs, we now have the two shapes that almost every program eventually wants: “all of these, together” and “exactly one of these”.

There are two more composites worth meeting before we leave the data chapters. Tuples give us “a fixed group of typed values without naming them”, and slices give us “a contiguous run of values of the same type”. Chapter 8 introduces both.

Chapter 8 - Tuples and Slices

Tuples Group Values Without Naming Them

A **tuple** is a fixed-length record whose components have types but no names. Tuples are useful when grouping values is worth the small bit of structure, but inventing field names would not pull its weight.

The type of a tuple is written with parentheses around a comma-separated list of types:

```
let pair: (i32, i32) = (3, 4);
let triple: (i32, bool, char) = (1, true, 'x');
```

A tuple expression has the same shape: parentheses around a list of values. The compiler checks that each component matches the declared type at its position.

Tuples are accessed by integer index using the same `.` notation we use for struct fields:

```
let first: i32 = pair.0;
let second: i32 = pair.1;
```

The numbers `0` and `1` are field names, not array indices. They refer to the first and second components of the tuple. There is no runtime bounds check involved, because the indices are part of the type.

A tuple type with two or more components is legal. There is no special form for a "one-tuple". When a function would naturally return one value, we return that value directly rather than wrapping it.

When To Reach For a Tuple

Tuples are tempting and easy to overuse. The honest test is whether the components have a relationship that a future reader will recognise without help. A pair of `(x, y)` coordinates, a pair of `(low, high)` bounds, a pair of `(value, count)` for a parsing helper — these all read clearly as tuples. A pair of "two unrelated configuration knobs" probably wants a struct with named fields.

When in doubt, prefer a struct. The cost of naming the fields is one line of declaration. The cost of leaving them anonymous is paid by every reader who has to remember which one was which.

Slices Describe a Run of Values

A **slice** is Rexy's way of describing a contiguous run of values of the same type. Where a tuple says "exactly these components, each with its own type", a slice says "some number of values, all of this one type". The slice itself does not own the underlying memory; it is a description of a run that lives somewhere else.

The slice type is written with `&` and square brackets around the element type:

```
fn sum(values: &[i32]) -> i32 {
    let mut total: i32 = 0;
    // ... iterate and accumulate ...
    return total;
}
```

A slice carries two facts about its underlying run: where it starts and how long it is. That makes a slice a natural way to write a function that processes a sequence without committing to where the sequence comes from. The same `sum` function above could be called with a slice that points at a static buffer, a slice that points into a heap region, or a slice that points at part of a larger run.

Slices work hand in hand with the pointer machinery we will introduce in Chapter 9. The pointer side gives us the address. The slice side gives us the length, and a uniform shape that functions can name in their signatures.

Where We Are by the End of Chapter 8

We can build records (structs), choices (enums), positional groups (tuples), and contiguous runs (slices). We have the shapes a working Rexy program needs to model the data it operates on.

What we have not yet seen is how to point *into* memory directly: how to take the address of a value, how to walk through a buffer, and how to mutate a static. Part IV introduces pointers, statics, and the `unsafe` boundary that surrounds them.

Part IV - Memory and the `unsafe` Boundary

Rexy is a systems language. That commitment becomes most concrete in Part IV, where we deal directly with addresses, with mutable storage that lives for the whole program, and with the rule that protects the rest of the language from those two capabilities.

Chapter 9 introduces pointers and statics together. Pointers give us a way to refer to a value by its address, dereference it, and walk through a run of values. Statics give us program-lifetime storage, both read-only scalars and mutable buffers. Chapter 10 introduces `unsafe`, the keyword that fences off the operations that need explicit programmer attention.

By the end of Part IV, the reader can describe and manipulate memory the way the operating system itself eventually has to. That is also the moment where Rexy demands the most discipline. We will spend the chapter explaining why, not just how.

Chapter 9 - Pointers and Statics

Why Rexy Has Pointers

Most modern languages hide pointers. They give us references, garbage collection, and value semantics, and they let us pretend that addresses do not exist. Rexy is a systems language and the operating system it serves ultimately has to manipulate memory by address. So pointers stay in the language, with a small but explicit syntax.

A **pointer** is a value whose contents are an address: it points at another value somewhere in memory. The type of a pointer is written `*T`, where `T` is the type of value the pointer is expected to point at:

```
let mut value: i32 = 7;
let p: *i32 = &value;
```

`&value` produces a pointer to `value`. The type of the pointer is `*i32` because `value` is an `i32`. The pointer is a value just like any other: we can pass it to a function, store it in a struct, or compare it to another pointer.

Reading and Writing Through a Pointer

To read the value a pointer points at, we **dereference** it with `*`:

```
let here: i32 = *p;
```

To write a new value through a pointer, we use `*p = ...`:

```
*p = 9;
```

The same `*p` form means "the value at the address `p`" on both sides of the assignment. On the right, it produces the value. On the left, it names the place the value will be written.

Pointers also support indexing. Given `p: *i32`, the expression `p[i]` reads the `i`-th `i32` value starting from `p`. Writing to `p[i] = ...` writes through that same offset. Indexing is exactly the same as `*(p + i)`, written more readably:

```
*(p + 0) = *p + 1;
let next: i32 = p[1 - 1];
```

Pointer arithmetic adds an integer offset, scaled by the size of the pointee type. `p + 1` advances by one `i32`, not by one byte.

When a Pointer Goes Wrong

Dereferencing an incorrect pointer is undefined: the value we read is whatever happens to be at the address, and the write we issue might land on memory that belongs to something else. Rexy does not check pointer arithmetic at runtime. The compiler does not promise that a `*i32` actually points at a live `i32`. That is part of the price of access at this level.

The rest of the language is built around values that carry their own meaning: a `bool` is always `true` or `false`, an enum is always one of its variants. Pointers do not give us that guarantee. That difference is why Rexy puts pointer use behind a fence, which we cover in Chapter 10.

`static` and `static mut`

A **static** is a value with program-lifetime storage. It exists from program start to program exit, regardless of what functions are running. Statics live at the top level of a file, alongside functions and types:

```
static SCALE: i32 = 100;
static mut COUNTER: i32 = 1;
```

A plain `static` is read-only. Other code can read its value but cannot assign to it. A `static mut` is mutable: the program may assign to it during execution. The mutation has the same explicitness rule as a `mut` local: if it is not labelled `mut`, it cannot be reassigned.

Statics are typed. The right-hand side of a static is a constant initial value. For scalar statics that value is just a literal. Statics also support a fixed-size buffer form, which is the systems-language equivalent of a global array:

```
static mut SCRATCH: [u8; 16];
static GREETING: [u8; 5] = [104, 101, 108, 108, 111];
```

The first declaration reserves a 16-byte buffer named `SCRATCH` whose initial contents are zero. The second declares a 5-byte buffer initialised to the byte values shown. The buffer name, used in an expression, evaluates to a pointer to its first element.

Pointer Use Lives Behind `unsafe`

We have shown a lot of pointer manipulation above without explaining the keyword most of those examples actually need: `unsafe`. The next chapter explains the rule and why it is the rule.

Where We Are by the End of Chapter 9

We have a way to refer to memory by address, to read and write through that address, to index across a run of typed values, and to declare program-lifetime storage that those pointers can target. Combined with slices from Chapter 8, we now have the full machinery a systems-language program uses to talk about memory.

What is missing is the explicit boundary the language draws between safe code and the operations that touch memory directly. Chapter 10 introduces `unsafe` and explains where it has to appear.

Chapter 10 - The `unsafe` Boundary

What `unsafe` Means

Rexy uses the keyword `unsafe` to mark code that performs operations whose correctness the compiler cannot check on its own. Inside an `unsafe` context, the programmer takes responsibility for promises the language itself does not enforce: that a pointer is valid, that an offset is in bounds, that a buffer is the right size, that a mutable static is being accessed at a moment when no other part of the program also needs it.

`unsafe` does not turn off the type system. Every type rule still applies. What `unsafe` does is permit a specific, named set of operations:

- Dereferencing a pointer (`*p` to read, `*p = ...` to write).
- Indexing through a pointer (`p[i]`, `p[i] = ...`).
- Pointer arithmetic (`p + n`, `p - n`).
- Reading or writing a `static mut`.
- Calling a function declared `unsafe fn`.

Outside an `unsafe` context, those operations are diagnostics. Inside one, they compile.

`unsafe fn`

A function may declare itself `unsafe`:

```
unsafe fn store(target: *i32, value: i32) -> i32 {
    *target = value;
    return value;
}
```

The body of an `unsafe fn` is itself an unsafe context. The function may freely use the operations listed above. The keyword in the signature is a contract: callers are being told that calling this function correctly requires more than the type system can confirm, so calling it must also happen in an unsafe context.

A safe function, declared without `unsafe`, may not perform unsafe operations. It also may not call an `unsafe fn`. The boundary is symmetric: producers of unsafe operations and consumers of unsafe functions both have to opt in.

`unsafe { }` Blocks

Inside an otherwise safe function, an `unsafe { }` block opens a localised unsafe context:

```
fn write_one(target: *i32) -> i32 {
    unsafe {
        *target = 1;
    }
    return 0;
}
```

The block is the smallest scope that needs to be unsafe. The surrounding function remains safe and can be called from anywhere. The unsafe operations are confined to the block.

Most pointer-using code in Rexy lives inside small `unsafe` blocks for exactly this reason. The function presents a safe surface to the rest of the program. The unsafe block is the visible spot where the programmer took responsibility for a memory operation.

A Concrete Example

The pointer manipulation we wrote in Chapter 9 needs an unsafe context to compile. With the boundary made explicit, the example reads:

```
unsafe fn pointers() -> i32 {
    let mut value: i32 = 7;
    let p: *i32 = &value;
    *p = 9;
    *(p + 0) = *p + 1;
    return p[1 - 1];
}
```

The function is `unsafe fn`, so its whole body is an unsafe context. Every dereference and every pointer-indexed access compiles because we are inside that context. Callers of `pointers` will have to be unsafe in turn.

Mutating a `static mut`

Mutable statics also live behind `unsafe`. Reading and writing them is one of the operations the boundary covers:

```
static mut COUNTER: i32 = 1;

unsafe fn bump() -> i32 {
    COUNTER = COUNTER + 1;
    return COUNTER;
}
```

The reasoning is the same as for pointers: the language cannot prove that a static mutation is well-timed with respect to whatever else might be running. The keyword keeps the operation visible.

Why The Rule Is Worth The Cost

Putting pointer access and `static mut` access behind `unsafe` does mean a small amount of extra ceremony. The trade is that every place memory could go wrong is named in the source. A reader auditing a Rexy program for memory bugs only has to read the unsafe contexts. The rest of the language is enforced by the type system.

That property gets more valuable, not less, as a program grows. The operating system that hosts our programs has to deal with memory failures constantly. The language that runs on top of it should not hide where those failures could come from.

Where We Are by the End of Chapter 10

We have the boundary that Part IV was building toward. Pointers and statics are real, usable, full-strength tools, and the contexts that use them are clearly marked. We can write low-level code without smearing unsafe operations across the whole program.

Part V steps back from memory and looks at how Rexy organises the source itself: modules, visibility, and generics. That is the toolkit we need to go from "a single source file with a `main`" to "a program that grows beyond one file".

Part V - Organising Programs

A real program rarely fits in one file. Sometimes it is too big. More often, some of its pieces are reusable, some are specific, and the codebase wants to draw the line between them. Rexy gives us two tools for that.

Chapter 11 introduces **modules**: how a Rexy file is split into named namespaces, how visibility works, and how `use` declarations bring names in. Chapter 12 introduces **generics**, the way Rexy lets us write a function or a struct once and reuse it across types.

By the end of Part V, the reader has the tools to grow a program past one file and to write data structures and algorithms that work for more than one specific type. After that, Part VI moves to the standard library and to running our programs on Drunix.

Chapter 11 - Modules and Visibility

Why Modules

Once a program has more than a few dozen items, putting them all in one file stops helping. We want to group related items together, give that group a name, and let the rest of the program refer to its contents through that name. A **module** is exactly that: a named namespace whose contents the program can reference.

Modules also serve a second purpose. They are the unit Rexy uses to draw visibility boundaries. By default, items are private to the module that declared them. Items that other modules need to use are marked `pub`.

Declaring a Module

There are two ways to declare a module in Rexy. The first is **inline**: the module's body sits directly inside braces:

```
mod arithmetic {
    pub fn add(a: i32, b: i32) -> i32 {
        return a + b;
    }
}
```

The second is **file-backed**: the module is declared with a semicolon, and its body lives in a file of the same name in the same directory:

```
mod math;
```

This declaration tells the compiler to look for `math.rx` next to the current source file and treat its contents as the body of the `math` module. File-backed modules are the form we will use most of the time. They keep each module to its own file and let the source tree mirror the module tree.

Qualified Names With `::`

Once a module exists, items inside it are reachable through a **qualified name**. The separator is `::`:

```
let result: i32 = math::add(20, 22);
let half: i32 = math::double(21);
```

Qualified names compose. A function inside a module inside a module is reached through two `::` separators. The compiler resolves the chain by walking the module tree from the current point, the same way a filesystem path resolves directories.

`use` Declarations

Writing `math::add` everywhere is fine for occasional calls. For code that reaches into the same module repeatedly, a `use` declaration brings the final name into the current scope:

```
mod math;
use math::add;

fn main() -> i32 {
    let imported: i32 = add(20, 22);
    let qualified: i32 = math::double(21);
    return imported + qualified - 42;
}
```

After the `use math::add;`, the bare name `add` refers to `math::add`. Qualified calls remain available; the `use` does not hide them. We pick whichever form reads better at the call site.

Visibility With `pub`

By default, a function, struct, enum, or static declared inside a module is private to that module. Code outside the module cannot call it, name it, or import it. To make an item visible to other modules, we mark it `pub`:

```
pub fn add(a: i32, b: i32) -> i32 {
    return a + b;
}

pub fn double(value: i32) -> i32 {
    return add(value, value);
}
```

The same `pub` keyword applies to structs, enums, and statics. A `pub struct` exposes the type and its fields. A `pub enum` exposes the type and its variants. A `pub static` exposes the static for read access (and write access too, if it is `static mut` and the caller is in an unsafe context).

Visibility lets a module decide what its outside surface looks like. Private helpers stay private. Public functions form the contract the rest of the program relies on. We reach for this distinction the moment a module grows beyond a single function.

Symbol Names Follow the Module Path

A function declared as `pub fn add` inside `mod math;` is known throughout the program as `math::add`. The compiler also derives a unique symbol name for it that incorporates the module path, so two `add` functions in two different modules do not collide at the linker level. The path-qualified language name and the unique symbol name are two views of the same thing.

Where We Are by the End of Chapter 11

We can split a Rexy program across multiple files, name the resulting modules, and choose what each module exposes. We can call into other modules through qualified names or through `use` declarations. We have enough structure now to grow a Rexy codebase without it becoming a single sprawling file.

What we still cannot do is write code that works for more than one specific type. The next chapter introduces generics and lets us reuse a function or a struct across the primitive types we already know.

Chapter 12 - Generics

Why Generics

Suppose we want a function that swaps the components of a pair. For `(i32, i32)`, the body is straightforward. For `(bool, bool)`, the body is the same, but with the type changed everywhere. Rexy lets us write that function once, with a placeholder name where the type used to be:

```
fn swap<T>(pair: (T, T)) -> (T, T) {
    return (pair.1, pair.0);
}
```

The angle brackets after the function name introduce a list of **type parameters**. Inside the body of the function, `T` is a type. Wherever a parameter, return type, or local declaration would name a concrete type, it can name `T` instead. When the function is called, the compiler picks a specific type for `T` and emits a copy of the body specialised for that type.

This is **monomorphization**: each distinct combination of type parameters becomes its own dedicated function in the final program. The user sees one generic function in the source, and the linker sees one specialised function per type the program actually uses.

Generic Functions

A generic function has its type parameters declared between the name and the parameter list:

```
fn first_or<T>(values: &[T], fallback: T) -> T {
    if len(values) > 0 {
        return values[0];
    }
    return fallback;
}
```

Inside the body, `T` is a single, consistent type for the duration of one call. A caller of `first_or` for a slice of `i32` and an `i32` fallback gets an `i32` back. A caller for a slice of `bool` and a `bool` fallback gets a `bool` back. The compiler enforces consistency at the call site: calling `first_or` with a `&[i32]` and a `bool` fallback is a diagnostic.

There is no syntax for naming the type parameter at the call site. Rexy infers each type parameter from the argument types and the expected return type. That inference is the one place the language does work the programmer does not have to write down. It is local to the call and the results are visible from the surrounding context.

Generic Structs

Structs may also take type parameters:

```
struct Pair<T> {
    first: T,
    second: T,
}
```

A `Pair<T>` is a struct with two fields of the same type. The type parameter `T` is part of the type's name. We say `Pair<i32>`, `Pair<bool>`, or `Pair<char>` whenever we need to refer to a specific instantiation.

Building a value uses the struct literal we already know, with the instantiation worked out from context:

```
let coords: Pair<i32> = Pair { first: 3, second: 4 };
let flags: Pair<bool> = Pair { first: true, second: false };
```

The annotation on the `let` binding pins the instantiation. Inside the struct literal, the fields take whatever type that instantiation names.

Multiple Type Parameters

A function or struct may take more than one type parameter. The list is comma-separated:

```
struct Either<L, R> {
    left: L,
    right: R,
}

fn first_of<L, R>(pair: Either<L, R>) -> L {
    return pair.left;
}
```

Each parameter is independent. A program may instantiate `Either<i32, bool>`, `Either<bool, i32>`, or `Either<i32, i32>`, and each is a distinct type.

Standard Library Types Are Generic

Two of the most useful types in the standard library are generic. `Option` represents an optional value, and `Result` represents a value that is either a success or a failure:

```
let maybe: Option<i32> = Some(42);
let outcome: Result<i32, MyErr> = Ok(7);
```

`Option<T>` has the variants `Some(T)` and `None`. `Result<T, E>` has the variants `Ok(T)` and `Err(E)`. Both are checked the same way as any other enum, with `match` arms that bind the carried value:

```
match maybe {
    Some(value) => { return value; },
    None => { return 0; },
}
```

The `?` operator is a small piece of syntax for propagating an `Err` up the call stack: applied to a `Result`, it unwraps an `Ok` value and returns the `Err` from the enclosing function on failure. We will use it sparingly in the book, but it is the standard idiom for chaining fallible calls.

What Monomorphization Costs

Each instantiation of a generic produces a separate compiled function. A program that calls `first_or` for both `&[i32]` and `&[bool]` ends up with two specialised copies in the final binary. That is the cost of erasure- free generics. The benefit is that every instantiation is fully type- checked and runs at the speed of code we would have written by hand.

For most programs the cost is small. For systems-language programs, where a small amount of duplication is cheap and a missing type guarantee is expensive, the trade is firmly worth it.

Where We Are by the End of Chapter 12

We can write functions and types once and use them across multiple concrete types. Combined with modules from Chapter 11, we now have the tools to organise a Rexy program of meaningful size and to share code within it.

Part VI takes everything we have built and turns it outward. The standard library gives us I/O, parsing, and string helpers. The final chapter walks the program from a `.rx` source file to a Drunix executable on disk.

Part VI - The Standard Library and Drunix

The previous five parts cover the language itself. This final part turns the language outward. Chapter 13 tours the standard library: the helpers Rexy makes available without ceremony for console I/O, string handling, and parsing. Chapter 14 closes the book by walking a Rexy program through the Drunix toolchain, from `.rx` source to a runnable ELF executable.

By the end of Part VI, the reader has met every piece of Rexy that this book is going to introduce. They can write a program, compile it, link it into a Drunix-compatible executable, and reason about how it interacts with the operating system that hosts it.

Chapter 13 - A Tour of the Standard Library

What the Prelude Gives Us

Rexy ships with a small set of functions that are available in every program without an explicit `use`. This is the **prelude**: the names that form the everyday vocabulary of Rexy code. The prelude focuses on the things almost every program needs: console I/O, integer formatting, string inspection, and a panic helper for unrecoverable errors.

We will tour the prelude by category. None of these functions need any import. They can be called from `main` and from any other function, just like a function we declared ourselves.

Console Output

Writing text to the console covers the largest share of stdlib calls in a typical program. The prelude provides a complementary pair of functions for plain text:

```
print("hello, ");
println("rexy");
```

`print` writes its argument to standard output without a trailing newline. `println` writes its argument followed by `\n`. Both take a `str`. Both return an `i32` status code, which most programs ignore.

For values other than strings, the prelude has typed helpers:

```
print_i32(42);
println_i32(-7);
print_bool(true);
println_bool(false);
print_char('!');
println_char('?');
```

Each helper formats its argument and writes it to standard output. Programs that need richer formatting compose these helpers with explicit casts and string concatenation built on slices and the underlying buffer helpers.

Console Input

Reading text from standard input uses two prelude functions:

```
let line: str = read_line();
let value: i32 = read_i32();
let flag: bool = read_bool();
```

`read_line` reads one line of input and returns it as a `str`. The trailing newline, if present, is stripped. The returned value points into a Rexy-owned buffer that is overwritten on the next call to `read_line`, so a program that needs to keep multiple lines should copy the contents to its own storage.

`read_i32` reads one line and parses it as a signed decimal integer. `read_bool` reads one line and parses it as `true` or `false`. Both fall back to default values for malformed input until the parsing helpers return typed results.

Parsing

Independent of input, the prelude exposes string-to-value parsers:

```
let value: i32 = parse_i32("-7");
let flag: bool = parse_bool("true");
```

`parse_i32` accepts an optional leading `-` followed by decimal digits. Empty strings, invalid characters, and overflow currently return `0`. `parse_bool` accepts the literal string `"true"` and returns `false` for anything else. Both behave the same way in scripted contexts and in interactive ones.

Strings

A small family of string predicates and helpers covers the most common inspection needs:

```
str_eq("rexc", "rexc");
str_starts_with("rexc", "re");
str_ends_with("rexc", "xc");
str_contains("rexc", "ex");
str_find("rexc", "ex");
str_is_empty("");
strlen("hello");
```

`str_eq` compares two strings for byte equality. `str_starts_with`, `str_ends_with`, and `str_contains` test for prefixes, suffixes, and substrings. `str_find` returns the byte index of a substring, or `-1` if the substring does not appear. `str_is_empty` reports whether a string has length zero. `strlen` returns the number of bytes before a null terminator.

These helpers operate on null-terminated byte strings. They give us enough machinery to check user input, dispatch on a flag, or build small text-driven utilities, without pulling in heavier string types.

Panicking

Some errors are unrecoverable. A program that encounters one calls `panic`:

```
panic("something went wrong");
```

`panic` writes `panic:` followed by the message, then terminates the program with status `101`. It returns `i32` for compatibility with expression contexts, but the program does not continue past the call.

Generic Stdlib Types

Two generic types from Chapter 12 deserve a second mention here, because they are part of the standard library surface even though their syntax came up earlier. `Option<T>` represents an optional value with variants `Some(T)` and `None`. `Result<T, E>` represents a fallible computation with variants `Ok(T)` and `Err(E)`. Both are matched the way any enum is matched. Both interact with the `?` operator for `Result` propagation through fallible call chains.

Beyond the Prelude

Names outside the default prelude are reachable through fully qualified paths such as `std::io::println` or `std::process::exit`. These bridge- backed declarations exist for code that wants explicit module discipline, or for compiler internals and tests that opt into broader access. For most programs, the prelude is enough.

Where We Are by the End of Chapter 13

We can read input, write output, parse a few common formats, inspect strings, and signal an unrecoverable error. Combined with everything from Parts I through V, we now have the whole language and the standard tools the language ships with.

The last thing we have not done is take a Rexy program and turn it into something the operating system can run. That is Chapter 14.

Chapter 14 - Building Programs for Drunix

From Source to Something the OS Can Run

Throughout the book we have written Rexy code without dwelling on what happens between saving a `.rx` file and seeing the program run. This final chapter walks that path. The goal is not a comprehensive build reference; the project's `README` covers every flag and target. The goal is an end-to-end picture of how Rexy source becomes a Drunix executable.

The Tools In Play

Three tools are involved between source and a runnable program:

- **`rexc`**, the Rexy compiler. It reads `.rx` files, type-checks them, and emits assembly, an object file, or a linked executable depending on what we asked for.
- **An assembler.** When `rexc` emits assembly, an assembler turns that text into an object file. `rexc` invokes one for us when we ask for object output or an executable.
- **A linker.** When the program needs to become a real executable, the linker combines our object with a startup object and the runtime library, using a linker script that describes the Drunix executable layout.

For everyday work we hand `rexc` a source file and an output path, and `rexc` orchestrates the rest.

The Smallest Useful Build

For a single-file program, the basic invocation produces a 32-bit Drunix ELF executable:

```
rexc hello.rx --target i386-drunix --drunix-root "$DRUNIX" -o hello.drunix
```

The flags are doing three things. `--target i386-drunix` selects the target triple: 32-bit Intel architecture, with the Drunix variant of the runtime and ABI. `--drunix-root` points at a Drunix checkout that contains the user-space linker script and the small startup object every Drunix executable needs. `-o` names the output file.

Behind that single command, `rexc` walks through the same pipeline a multi-tool build would walk through by hand: it produces an assembly representation of the program, assembles it into an object, emits a small Drunix startup object that provides `_start` and prepares `argc`, `argv`, and `envp`, includes the target-specific hosted runtime, and invokes the linker with the Drunix user linker script. The result is a runnable executable.

What `_start` and `main` Do Together

Drunix expects every user program to begin at a symbol called `_start`. `_start` is provided by the startup object `rexc` emits as part of the build. Its job is to set up the environment the program expects, call the user's `main`, and then exit cleanly through Drunix's syscall adapter.

Our Rexy `main` function returns an `i32`. The startup code reads that return value, hands it to the operating system as the program's exit status, and terminates the process. The convention from Chapter 1 lines up with this: returning `0` says success, and any other value says something the operating system or the parent process can interpret.

Selecting Other Targets

The same compiler binary can target other architectures. The 64-bit Linux target produces ELF64 objects:

```
rexc wide.rx --target x86_64 -c -o wide64.o
```

The `-c` flag produces an object file rather than an executable. Linking that object into a runnable program requires an `x86_64` startup, runtime, and linker script that match the host's calling convention. The Drunix user runtime is currently focused on `i386-drunix`, so the 64-bit object form is most useful as part of a separate build process or as a way to inspect the compiler's emitted code on a different target.

There is also a Darwin ARM64 target useful during development on Apple silicon hosts. The flag is `--target arm64-macos`. The mechanics are the same as above: the compiler emits target-appropriate assembly and links against the matching runtime.

Inspecting the Result

Because Rexy programs become standard ELF executables, the same tools we use to inspect any ELF file work here. `file` confirms the binary class and architecture:

```
file hello.drunix
```

`readelf -h` shows the executable header, including the entry point and the architecture:

```
x86_64-elf-readelf -h hello.drunix
```

For a 32-bit Drunix executable the header reports `ELF32` and `Intel 80386`, with the entry point set to the address of `_start`. Running the executable on Drunix produces the program's output, exits with the value `main` returned, and rejoins the operating system's run queue like any other process.

Where We Are by the End of the Book

We have walked from "I have never written a Rexy file" to "I can compile a multi-file Rexy program with structs, enums, generics, modules, pointers, and stdlib calls into a runnable Drunix executable". Along the way we have met every construct the language currently provides and seen what each one is for.

The compiler's internal pipeline – how `rexc` actually performs lexing, parsing, semantic analysis, IR lowering, code generation, and linking – is its own subject and not the subject of this book. The README and the compiler source are the right places to follow that thread further. What this book has tried to give us is the part that comes first: a working Rexy programmer.

From here, the way to learn more is to write Rexy. Pick a small problem. Write it as a Rexy program. Run it on Drunix. The language is small enough that a few weeks of practice will let it disappear into the problem, which is the point at which a programming language has done its job.